

Benchmark de Algoritmos de Ordenação

Édipo Elder Fernandes de Melo

04/10/2026

1 Introdução

Este relatório apresenta uma análise comparativa de cinco algoritmos de ordenação aplicados a uma estrutura de dados em C++ (`std::vector<int>`). O objetivo é medir, de forma padronizada, o tempo de execução médio (em milissegundos) de cada algoritmo em função do tamanho do vetor, considerando três cenários distintos: melhor caso, pior caso e caso médio.

O melhor caso é quando o vetor já está ordenado. O pior caso é quando o vetor está ordenado de forma decrescente. Para testar o caso médio, optamos em criar um vetor com valores aleatórios. Neste último caso de cenário, é possível que o vetor tenha valores repetidos.

1.1 Os algoritmos avaliados

A seguir, resumiremos as características de cada algoritmo testado:

Insertion Sort. Percorre o vetor da esquerda para a direita, inserindo cada elemento na posição correta dentro da parte já ordenada. É eficiente para vetores pequenos ou quase ordenados, mas quadrático no pior caso.

Selection Sort. A cada iteração, seleciona o menor elemento da parte não ordenada e o coloca na posição correta. Realiza sempre o mesmo número de comparações, independentemente da ordem inicial do vetor.

Bubble Sort. Compara pares adjacentes e troca-os se estiverem fora de ordem, repetindo o processo até que nenhuma troca seja necessária. Possui uma otimização que interrompe a execução precocemente se o vetor já estiver ordenado.

Quick Sort. Escolhe um pivô (aleatório, neste experimento), particiona o vetor em torno dele e ordena recursivamente as subpartes. É um dos algoritmos mais eficientes na prática, com complexidade média $O(n \log n)$.

Merge Sort. Divide o vetor recursivamente pela metade, ordena cada metade e depois as intercala. Garante complexidade $O(n \log n)$ em todos os casos, mas utiliza memória auxiliar.

A tabela a seguir resume a complexidade dos algoritmos no melhor, pior e caso médio:

Tabela 1: Complexidade dos algoritmos de ordenação avaliados

Algoritmo	Melhor	Médio	Pior	Estável?
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Sim
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	Não
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Sim
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	Não
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Sim

1.2 Cenário do experimento

O programa foi executado num processador Apple M4, compilado com Apple clang versão 21.0.0 (clang-2100.3.34.2) e os seguintes parâmetros:

```
g++ -std=c++17 -O2 benchmark_sort.cpp -o bench_sort
```

Em todos os testes:

- Foram criados três vetores com tamanho `VECTOR_SIZE` (15000), utilizados de forma padrão nos testes, sendo preenchidos de formas distintas:
 - **Melhor caso:** vetor já ordenado (`melhor_caso[i] = i`).
 - **Pior caso:** vetor em ordem inversa (`pior_caso[VECTOR_SIZE - i - 1] = i`).
 - **Caso médio:** vetor aleatório (`rand() % VECTOR_SIZE`).
- Os tamanhos de vetor testados variam de 1000 a 15000, em passos de 1000.
- Cada medição foi repetida 3 vezes (`TRIALS`), e calculamos a média dos tempos obtidos.
- Para evitar que o algoritmo receba um vetor já ordenado nos cenários de testes do pior caso e caso médio, a cada nova execução do algoritmo foi criada uma cópia do vetor padrão.

2 A função de benchmark

A fim de evitar repetição de código para cada algoritmo implementado, padronizamos a execução dos testes com um função genérica:

```
typedef void (*algoritmo_ordenacao)(vector<int>& vetor, int end);

inline long benchmark(algoritmo_ordenacao algoritmo, vector<int> &vetor, int end) {
    ...
}
```

A implementação dos algoritmos foi feita:

1. **Com a mesma assinatura de parâmetros.** Todas as funções recebem (`vetor`, `end`) e retornam `void`. O ponteiro de função `algoritmo_ordenacao` permite chamá-las polimorficamente.
2. **Medição isolada.** O cronômetro só envolve a chamada do algoritmo em si. As instruções para criação da cópia do vetor padrão não são computadas no teste.
3. **Cópia do vetor a cada teste.** Isso evita que o algoritmo receba um vetor já ordenado na execução seguinte.

O `main` cria os três vetores de teste e, para cada função registrada na tabela de `funcoes`, computa a média de tempo de 3 repetições para diferentes tamanhos, gerando um arquivo CSV diretamente utilizável pelo R. Optamos por fazer uma média de 3 execuções uma vez que os algoritmos Bubble Sort e Selection sort levam de 25 a quase 200 segundos de execução.

3 Análise por algoritmo

3.1 Insertion Sort

No melhor caso (vetor já ordenado), o algoritmo percorre o vetor uma única vez, resultando em crescimento linear. No pior caso (ordem inversa) e no caso médio, o comportamento é quadrático, com o pior caso apresentando tempos significativamente maiores.

```
void insertion_sort(vector<int>& vetor, int end){
    int key, holePos;
    for (int i = 1; i < end; i++) {
        key = vetor[i];
        holePos = i - 1;
        while (holePos >= 0 && vetor[holePos] > key) {
            vetor[holePos + 1] = vetor[holePos];
            holePos = holePos - 1;
        }
        vetor[holePos + 1] = key;
    }
}
```

```

    }
    vetor[holePos + 1] = key;
}

```

Insertion sort

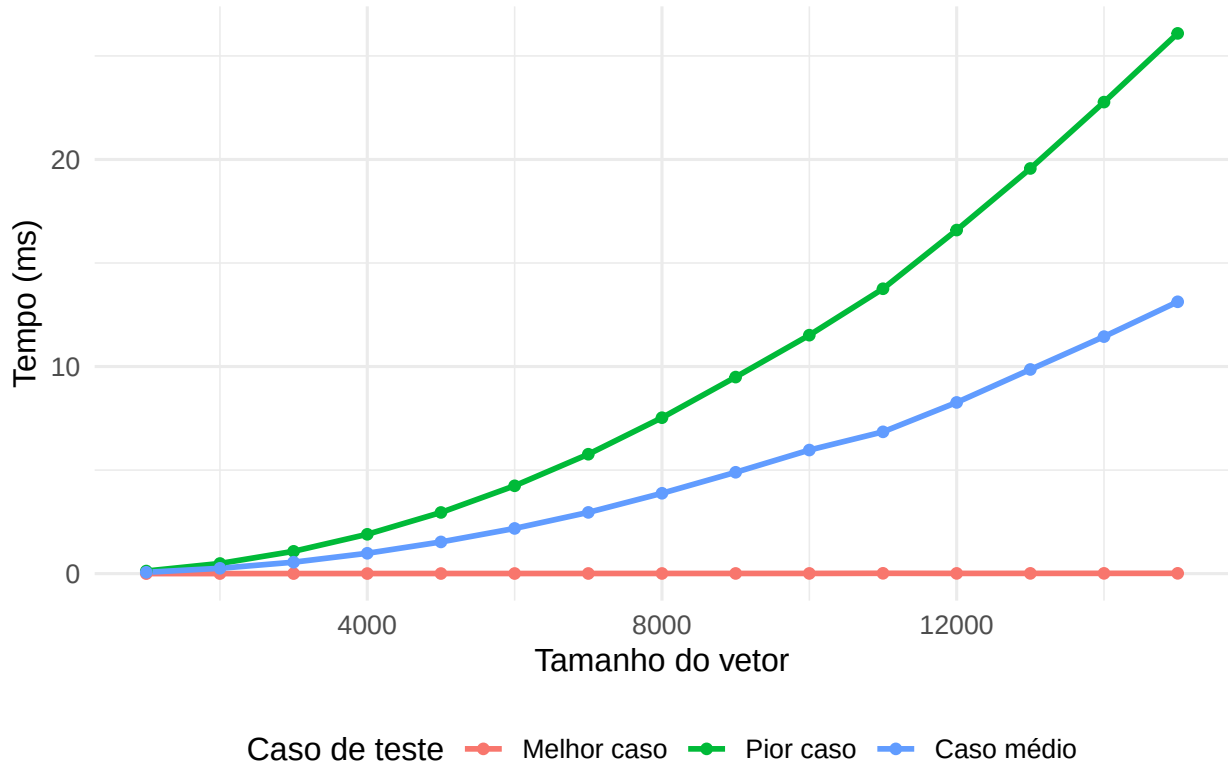


Figura 1: Insertion Sort

3.2 Selection Sort

O Selection Sort realiza sempre o mesmo número de comparações, independentemente da ordem inicial. Contudo, a instrução que determina a troca de valores no vetor (`if (vetor[i] > vetor[j])`) gera um impacto maior quando o vetor contém dados aleatórios, conforme pode ser observado na Figura 2.

```

void selection_sort(vector<int>& vetor, int end){
    for (int i = 0; i < end - 1; i++)
        for (int j = i + 1; j < end; j++)
            if (vetor[i] > vetor[j])
                swap(vetor[i], vetor[j]);
}

```

3.3 Bubble Sort

O Bubble Sort com a otimização de parada precoce apresenta comportamento linear no melhor caso (vetor já ordenado). Nos demais casos, o crescimento é quadrático. AS “quebras” no gráfico do melhor caso podem ser consequência da execução concorrente com outros programas do sistema operacional.

```

void bubble_sort(vector<int>& vetor, int end){
    int j = end - 1;
    bool houveTroca = false;
    do {
        houveTroca = false;
        for (int i = 0; i <= j; i++) {

```

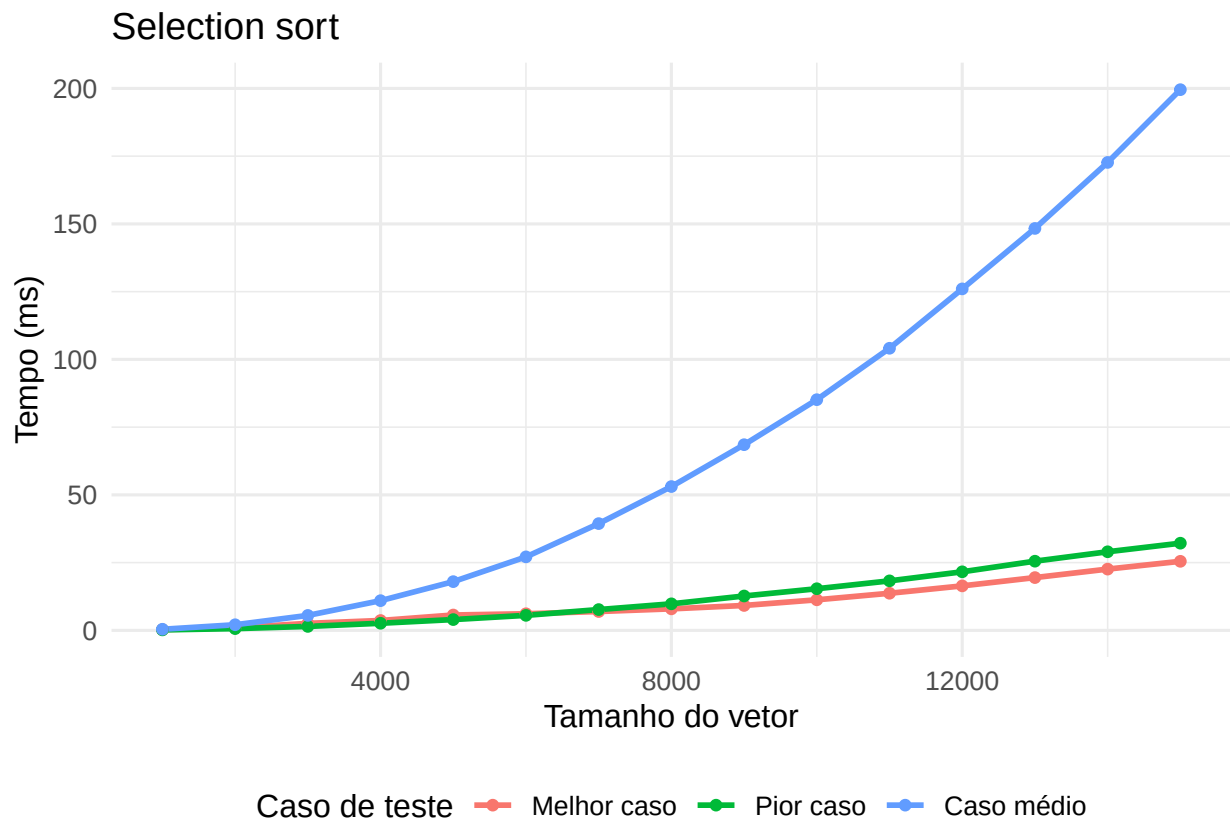


Figura 2: Selection Sort

```

        if (vetor[i + 1] < vetor[i]) {
            swap(vetor[i], vetor[i + 1]);
            houveTroca = true;
        }
    }
    j = j - 1;
} while (houveTroca);
}

```

3.4 Quick Sort

O Quick Sort apresenta comportamento $O(n \log n)$ em todos os casos. Nesta implementação, optamos escolher um pivô aleatoriamente. Observa-se que o pior caso teórico (vetor já ordenado) não se manifesta de forma significativa, e as três curvas ficam próximas. Os tempos obtidos no caso médio podem decorrer da excessiva troca de valores no vetor para valores iguais.

```

int particao(vector<int>& vetor, int start, int end){
    int pivô = start + rand() % (end - start + 1);
    int x = vetor[pivô];
    swap(vetor[pivô], vetor[end]);
    int i = start - 1;
    for (int j = start; j < end; j++) {
        if (vetor[j] <= x) {
            i++;
            swap(vetor[i], vetor[j]);
        }
    }
    swap(vetor[i + 1], vetor[end]);
    return i + 1;
}

```

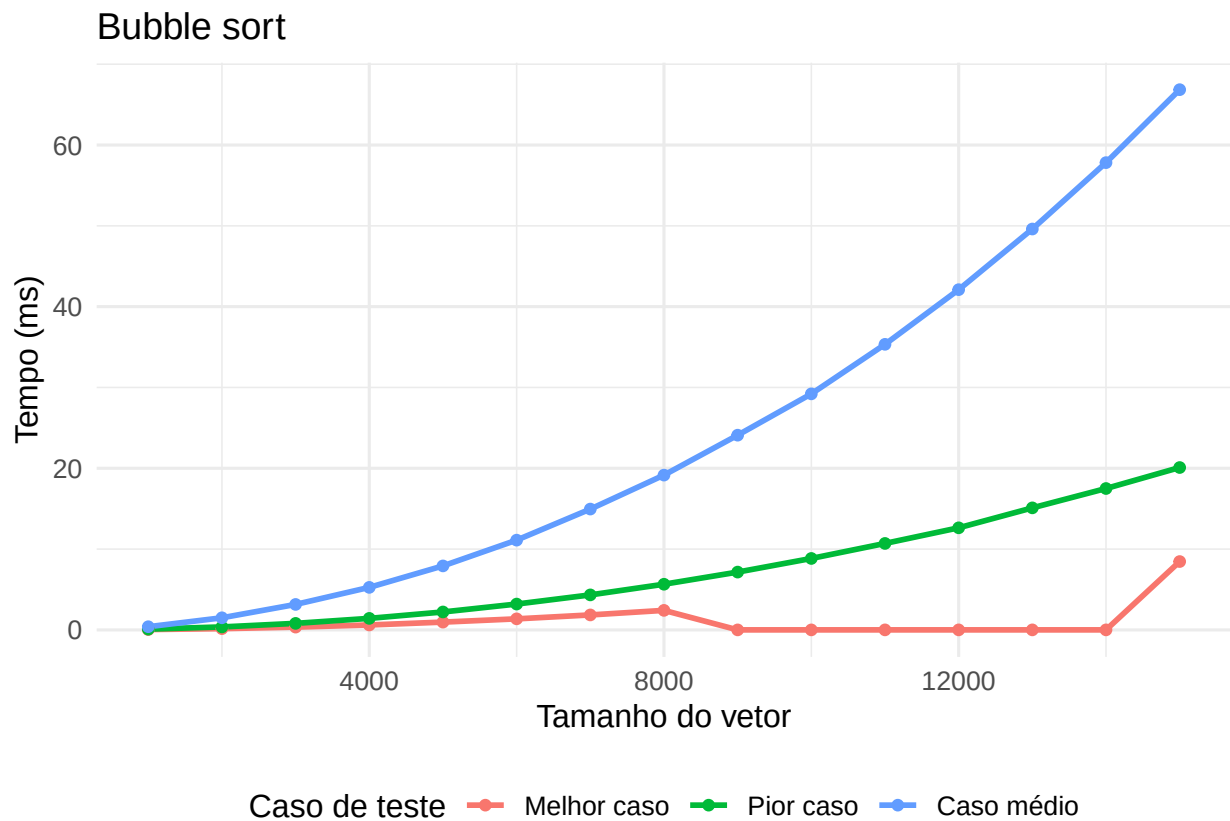


Figura 3: Bubble Sort

```

}

void quick_sort(vector<int>& vetor, int start, int end){
    if (start < end) {
        int idx = particao(vetor, start, end);
        quick_sort(vetor, start, idx - 1);
        quick_sort(vetor, idx + 1, end);
    }
}

void quick_sort(vector<int>& vetor, int end){
    quick_sort(vetor, 0, end);
}

```

3.5 Merge Sort

O Merge Sort também apresenta comportamento $O(n \log n)$ em todos os casos, com curvas praticamente coincidentes. Seu tempo é ligeiramente superior ao do Quick Sort devido ao uso de memória auxiliar. De mesma forma, para o caso médio, a curva de tempo de execução é ligeiramente superior.

```

void intercala(vector<int>& left, vector<int>& right, vector<int>& vec) {
    int i = 0, j = 0, k = 0;
    while (i < left.size() && j < right.size()) {
        if (left[i] <= right[j]) vec[k++] = left[i++];
        else vec[k++] = right[j++];
    }
    while (i < left.size()) vec[k++] = left[i++];
    while (j < right.size()) vec[k++] = right[j++];
}

```

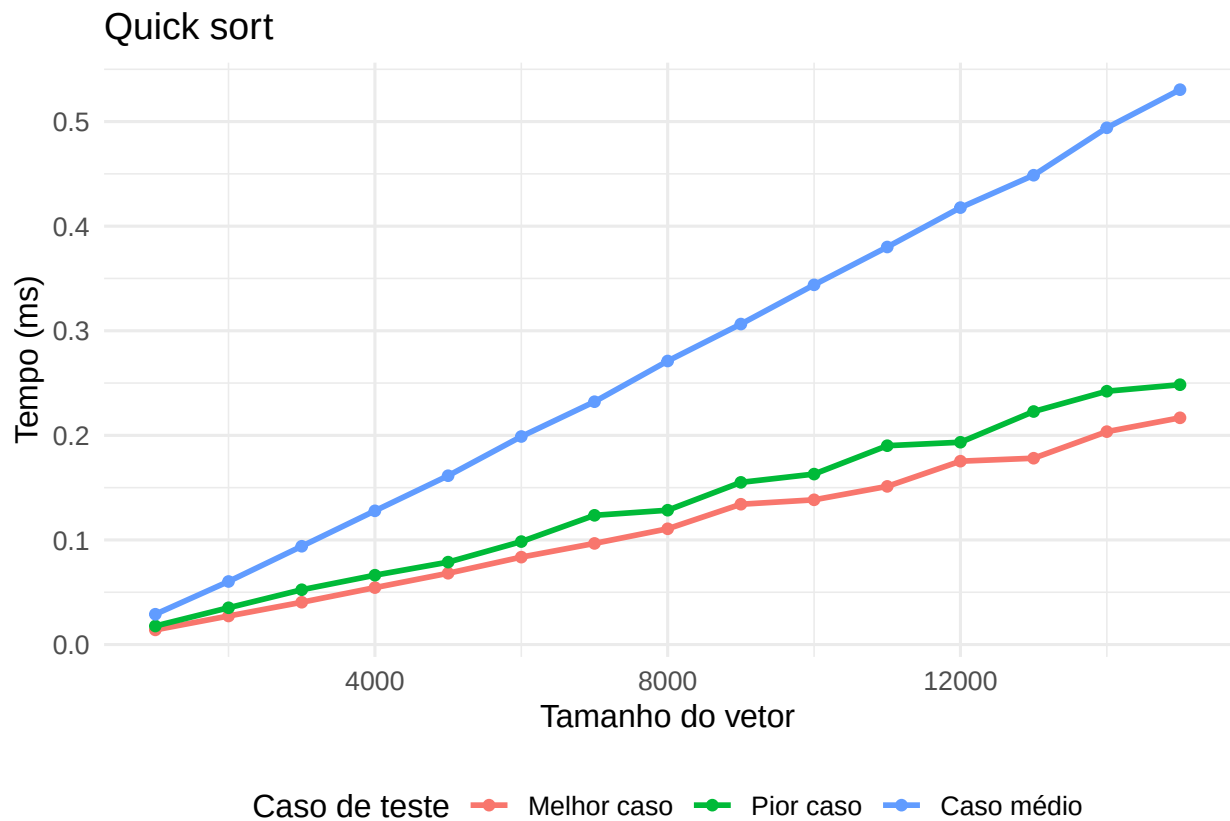


Figura 4: Quick Sort

```
void merge_sort(vector<int>& vetor, int end) {
    if (end < 2) return;
    int meio = end / 2;
    vector<int> left(meio);
    vector<int> right(end - meio);
    for (int i = 0; i < meio; i++) left[i] = vetor[i];
    for (int i = meio; i < end; i++) right[i - meio] = vetor[i];
    merge_sort(left, meio);
    merge_sort(right, end - meio);
    intercala(left, right, vetor);
}
```

4 Conclusão

4.1 Melhor caso (vetor já ordenado)

No melhor caso, Insertion Sort e Bubble Sort apresentam crescimento linear, enquanto Selection Sort, Quick Sort e Merge Sort mantêm seus comportamentos característicos. O Insertion Sort é o mais rápido nesse cenário.

4.2 Pior caso (vetor em ordem inversa)

No pior caso, Insertion Sort, Selection Sort e Bubble Sort apresentam crescimento quadrático acentuado, com tempos muito superiores aos do Quick Sort e Merge Sort, que se mantêm em $O(n \log n)$.

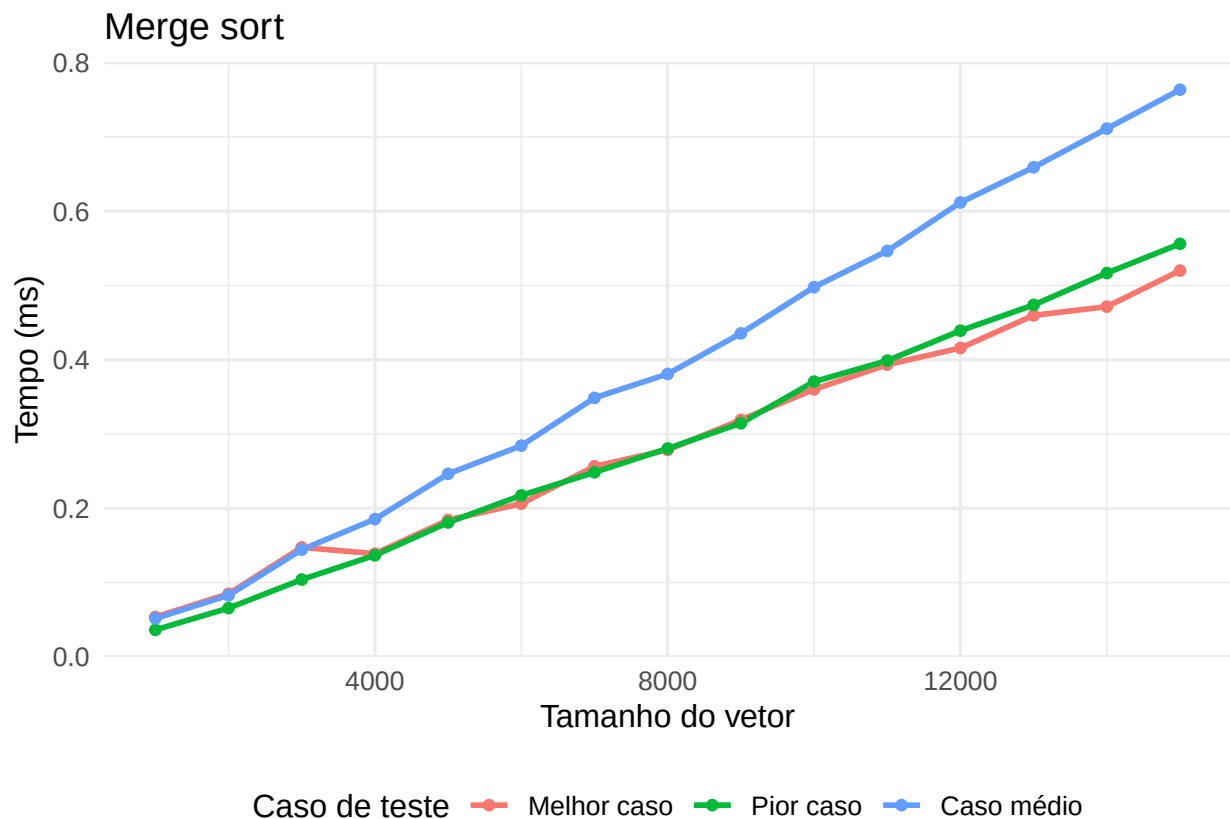


Figura 5: Merge Sort

4.3 Caso médio (vetor aleatório)

No caso médio, o comportamento é semelhante ao pior caso para os algoritmos quadráticos, mas com tempos ligeiramente menores. Quick Sort e Merge Sort continuam sendo as melhores escolhas para vetores grandes.

4.4 Considerações finais

A medição com a biblioteca `chrono` não é ideal, uma vez que o sistema operacional pode flutuar a execução do programa. Isso foi observado em experimento anterior, com algoritmos de busca, e ao executar este programa diversas vezes, obtendo gráficos ligeiramente distintos.

Concluindo a análise, observamos que, para vetores grandes e desordenados, o **Quick Sort** e o **Merge Sort** são as melhores escolhas, com complexidade $O(n \log n)$ em todos os casos. O **Insertion Sort** se destaca no melhor caso (vetor já ordenado) com comportamento linear, sendo útil quando os dados estão quase ordenados. Já o **Selection Sort** e o **Bubble Sort**, embora sejam de simples implementação, são ineficientes para vetores grandes, apresentando crescimento quadrático mesmo no melhor caso (no caso do Selection Sort).

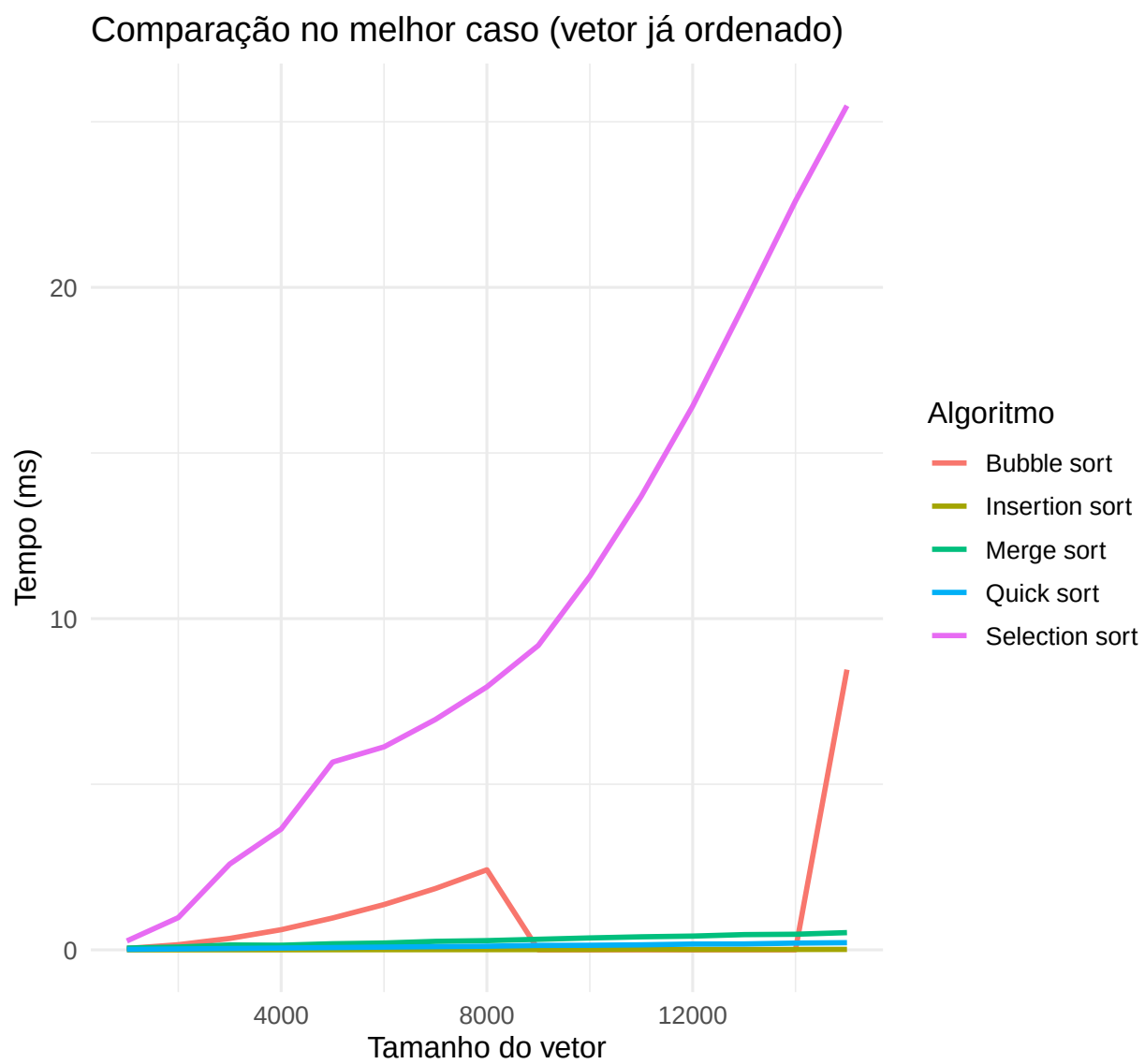


Figura 6: Comparação no melhor caso

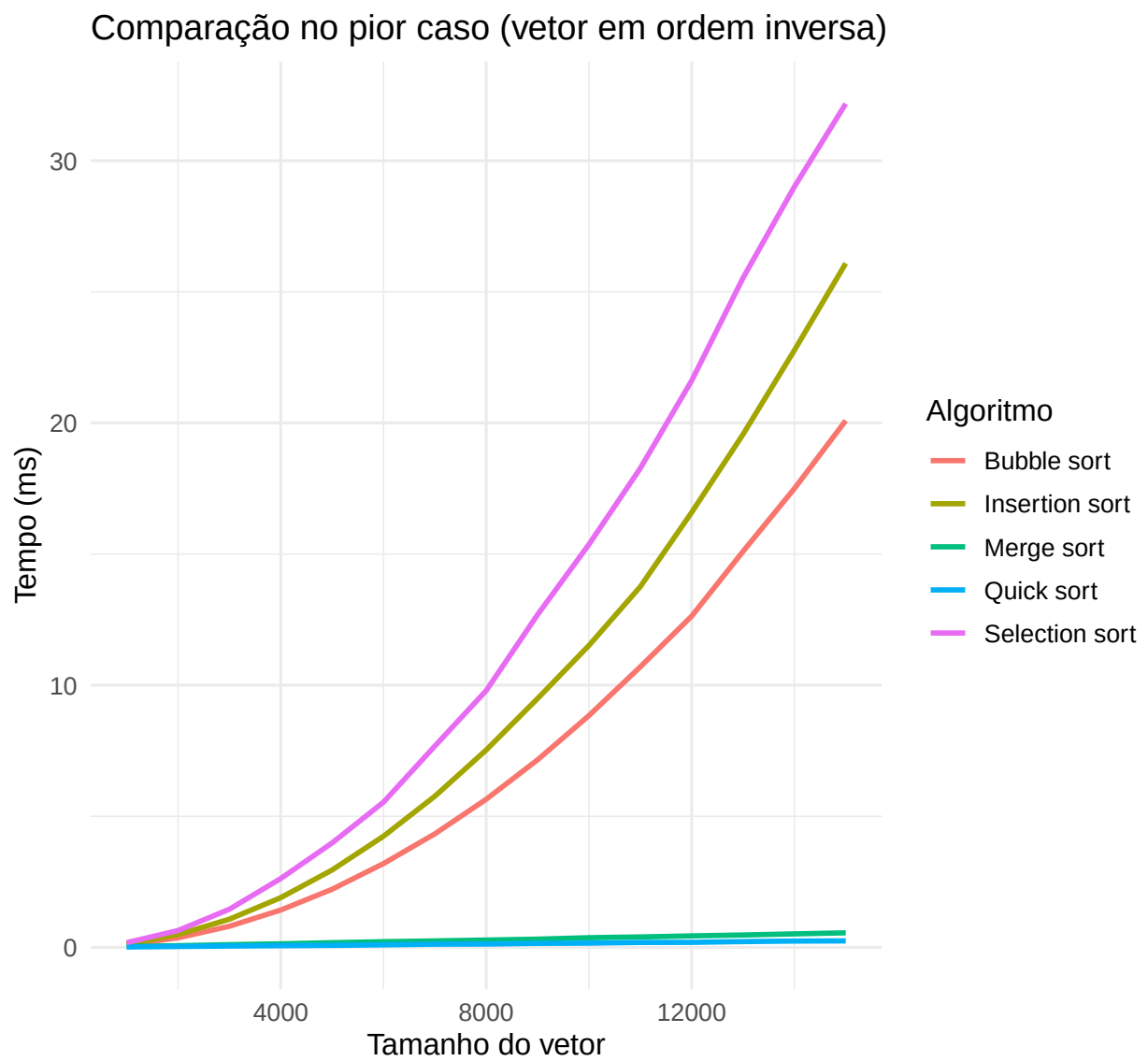


Figura 7: Comparação no pior caso

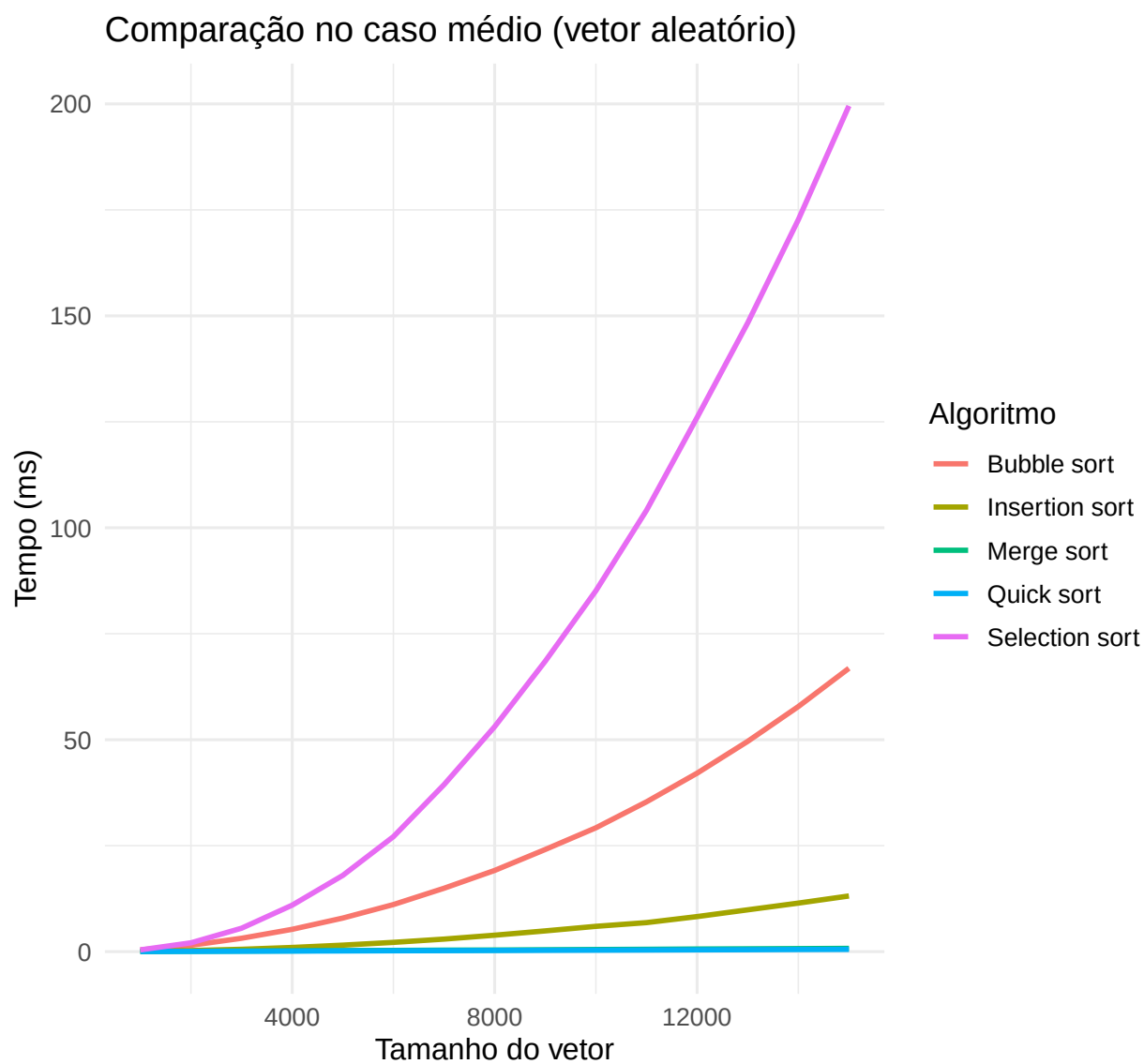


Figura 8: Comparação no caso médio

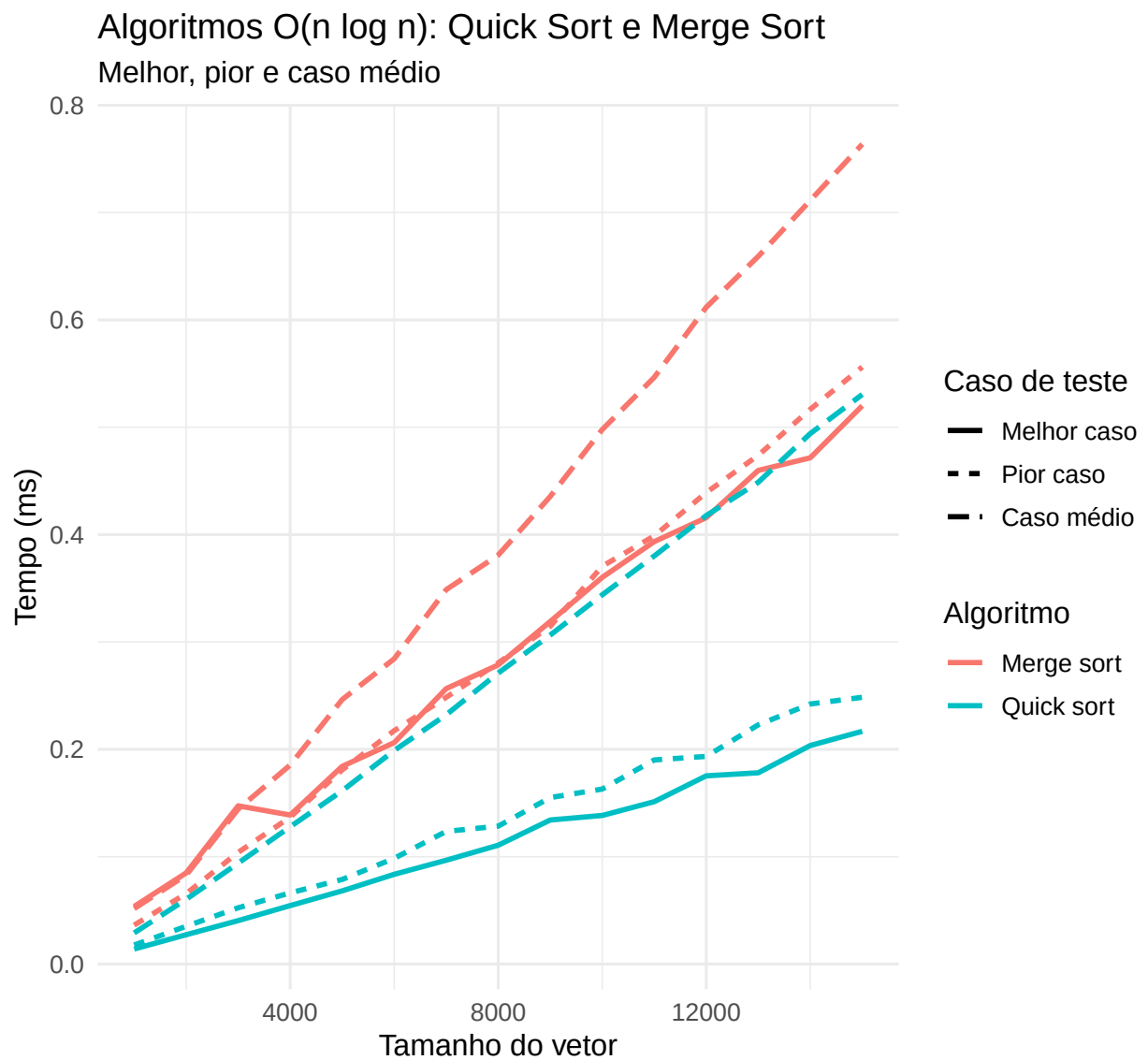


Figura 9: Comparação entre os algoritmos $O(n \log n)$

Demais algoritmos: Insertion, Selection e Bubble Sort

Melhor, pior e caso médio

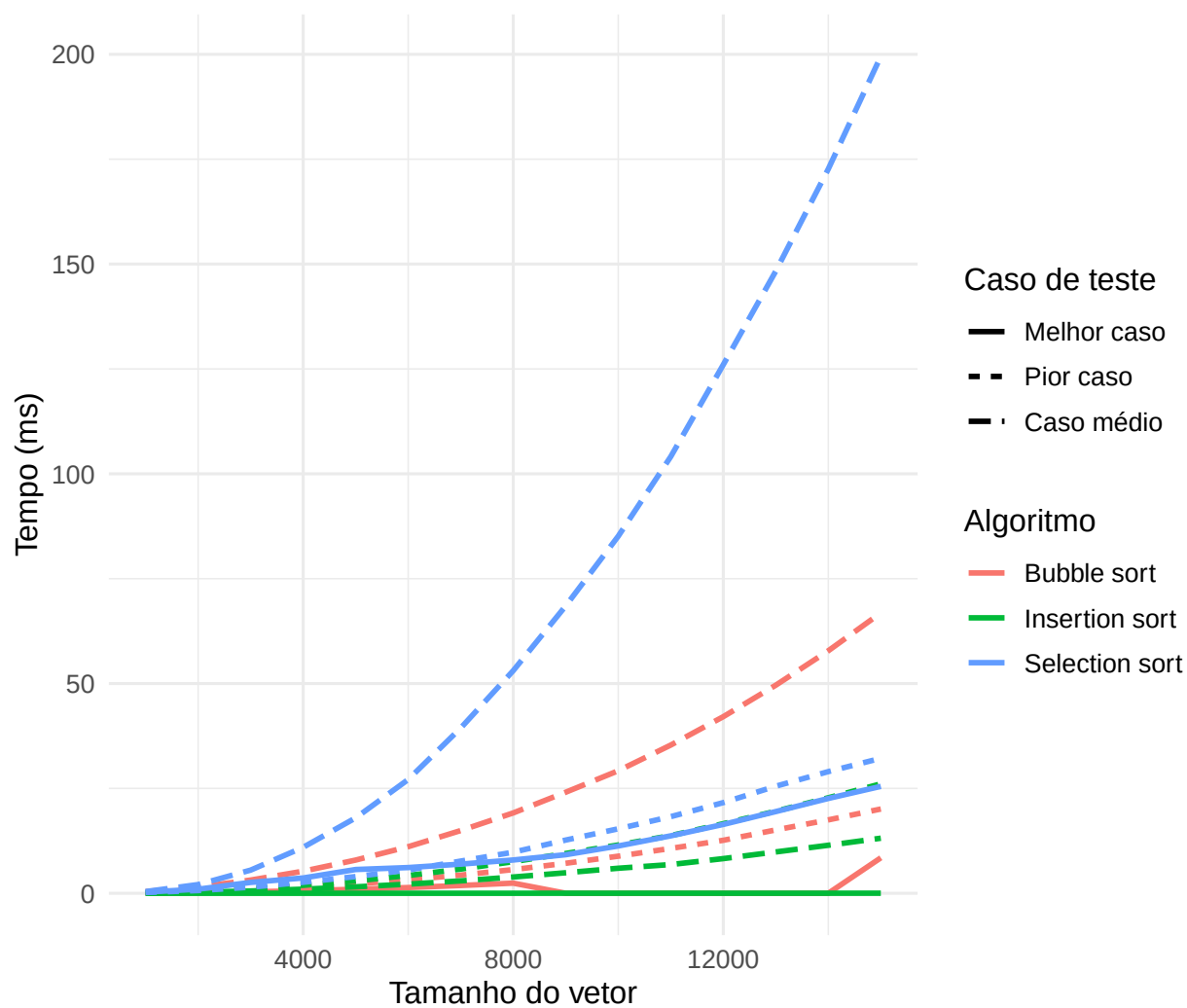


Figura 10: Comparação entre os demais algoritmos